AI Job Search and Career Agent

Stage 2 Report: AI-Assisted Implementation

EECS3311 | Zarak Khan, York University | 222046965 | November 2026

Repository: https://github.com/zarakkhancs/AI-Job-Search-and-Career-Agent

1. Project overview and implementation status

Application and users. The AI Job Search and Career Agent is for software engineering students and early-career developers who apply to co-ops, internships and new-grad roles. It helps a job seeker extract and match skills, tailor application documents, track applications through a pipeline, get deadline alerts, practise interviews and review analytics. Stage 2 implements the Stage 1 design (12 features, F01 to F12) in Java, with a Next.js GUI and a CLI over one shared backend.

AI/LLM integration. The Gemini API (model gemini-3.5-flash-lite) sits behind an `LLMClient` interface. The LLM is used for skill extraction, tailored summaries and cover letters, the optional LLM match strategy, bullet rewrites, prep sheets, mock interview questions and feedback, and agent planning and answer writing. Everything else (job ingestion, the application pipeline, deadline alerts, analytics, export and all source verification) is deterministic code. The API key is read from a local, git-ignored `.env`; `.env.example` holds placeholders only.

Architecture as built

- **Backend:** Java 25, Maven, package `com.careeragent`. All business logic is in services created once in `AppContext` and shared by the CLI and the HTTP API.
- **CLI:** Command pattern, one class per command (12 commands) registered in `CommandRegistry`.
- **HTTP API:** Javalin REST layer (`ApiServer`, port 7070) with consistent error mapping (400 bad input, 404 not found, 409 illegal state change, 502 AI failure).
- **GUI:** Next.js, React and Tailwind in `frontend/`, calling the API through a `/api` rewrite. Pages: dashboard (deadlines, alerts), resume, jobs, skills, tailor, pipeline (with analytics), bullets, prep, interview, agent.
- **AI access:** `LLMClient` with `GeminiClient` for production and `MockLLMClient` for tests (Figure 8). Figure 1 shows the overall class structure.

Current working functionality

All 12 Stage 1 features are implemented and reachable from both the GUI and the CLI (F03 and F04 from the CLI through the `agent` and `export` commands). 230 automated JUnit tests pass with `mvn test`; they use `MockLLMClient` and Javalin test tools, so they are deterministic and need no API key. Setup, configuration and run commands are in the repository README.

Limitations and incomplete functionality

- Data is kept in memory (applications, alerts, deadlines, resumes, jobs, interview sessions) and resets when the backend restarts. Stage 1 planned PostgreSQL.
- F01 extracts skills only, not work experience or education.
- F04 has no tone option. F05 moves applications with buttons, not drag-and-drop. F09 shows counts and rates, not charts. F10 exports plain text, not PDF.
- The agent (F11) chooses one tool per request. It does not build multi-step plans, replan, keep memory or ask clarifying questions as Stage 1 described.
- Deadline alerts are created when "Check deadlines" is pressed (or the API is called), not by a background scheduler. The GUI reads unread alerts; nothing is pushed to it.
- Factual checks cover skills and numeric figures only. Employers, degrees and dates are not verified.
- Only two of the seven planned prompt factories were built; the other prompts are assembled inside their services.

- Some job sites block automatic downloads, so pasting the posting text is the fallback. The free Gemini tier sometimes returns 429 or 503 errors or non-JSON text; the client retries and a retry in the UI usually works.

2. Feature to implementation mapping

Every Stage 1 feature (F01 to F12) and every addition is listed. Class names are in package `com.careeragent`. The video column gives the file and where the GUI and CLI parts start. "Stage 1 design" means the Stage 1 report in `stage1/`.

F01 Resume Import & Parsing

Stage 1 reference	F01 (UC-01, SD-01). Stage 1 patterns: Strategy, Facade, Factory Method, Adapter, Command.
Status / change	**Modified.** Reads a PDF or pasted text and extracts skills only; no experience or education; stored in memory. See Section 4.
Type	Hybrid: text extraction and verification are deterministic, skill extraction is by the LLM.
Classes and responsibilities	`PdfTextExtractor`: reads the text of a PDF (Apache PDFBox). `ResumeImportService`: orchestrates the import and keeps the list of `ParsedResume` records. `SkillExtractor`: builds the prompt, calls `LLMClient`, parses the JSON list, retries once if no JSON array comes back. `SourceVerifier` and `ExtractionResult`: split extracted skills into verified and rejected. `ResumeCommand` (CLI), `ApiServer` (`POST /api/resumes`, `POST /api/extract`), GUI `/resume`.
Key methods	`PdfTextExtractor.extract(byte[])`, `ResumeImportService.importPdf(filename, bytes)`, `importText(name, text)`, `SkillExtractor.extract(text)`, `SourceVerifier.isSupported(claim, source)`
Design patterns	Adapter (LLM access), Command (CLI).
Collaboration	The Resume page or `resume` command sends a PDF or pasted text to `ResumeImportService`. For a PDF, `PdfTextExtractor` returns the text. `SkillExtractor` asks the LLM for skills, then `SourceVerifier` keeps only skills that appear in the text and reports the rest as rejected. The service stores a `ParsedResume` and returns it.
Video	F01-resume-import.mp4 GUI from 0:00, CLI from 0:25

F02 Job Posting Ingestion

Stage 1 reference	F02 (UC-02, SD-02). Stage 1 pattern: Strategy (`ExtractionStrategy`), Command.
Status / change	**Modified.** Imports from a URL or pasted text and lists postings on the Jobs page. The Strategy classes were replaced by a `PageFetcher` interface and a small branch. See Section 4.
Type	Deterministic (no AI).
Classes and responsibilities	`JobIngestionService`: decides whether the input is a URL or text, stores a `JobPosting`. `PageFetcher` / `HttpPageFetcher`: downloads a page; only http and https are allowed and local or private addresses are refused. `HtmlText`: turns HTML into plain text. `JobCommand` (CLI), `ApiServer` (`/api/jobs`), GUI `/jobs`.
Key methods	`JobIngestionService.ingest(input)`, `ingestText(source, text)`, `list()`, `get(id)`, `HttpPageFetcher.fetch(url)`
Design patterns	Command (CLI).

Collaboration	The Jobs page or `job` command passes the input to `JobIngestionService.ingest`. A URL goes through `HttpPageFetcher.fetch` and `HtmlText`; pasted text is used directly. The result is saved as a `JobPosting(id, source, text)`. If a site blocks the download, the user pastes the text instead.
Video	F02-job-ingestion.mp4 GUI from 0:00, CLI from 0:37

F03 Skill Gap & Match Analysis

Stage 1 reference	F03 (UC-03, SD-03). Stage 1 patterns: Facade, Factory Method, Adapter, Command.
Status / change	**Modified.** Score is the percentage of job skills found in the resume. Matching is a Strategy with a keyword strategy (default) and an LLM strategy.
Type	Deterministic by default; AI-based when `LlmMatchingStrategy` is selected.
Classes and responsibilities	`JobMatcher`: context; holds the current `MatchingStrategy`. `KeywordOverlapStrategy`: counts matched and missing skills (uses `SkillComparison`). `LlmMatchingStrategy`: asks the LLM to judge the match. `MatchResult(score, matched, missing)`, `MatchJobTool` (agent tool), `ApiServer` (`POST /api/match`), GUI `/skills`.
Key methods	`JobMatcher.setStrategy(strategy)`, `JobMatcher.match(resumeSkills, jobSkills)`, `MatchingStrategy.match(...)`
Design patterns	Strategy, Adapter (LLM strategy), Facade (through the agent).
Collaboration	The Skills page posts the resume skills and job skills to `/api/match`. `JobMatcher` delegates to its current strategy, which returns a `MatchResult` with the score and the matched and missing skills. In the CLI this is reached through the `agent` command (`match_job` tool).
Video	F03-skill-match.mp4 GUI from 0:00, CLI from 0:42

F04 Tailored Cover Letter

Stage 1 reference	F04 (UC-04, SD-04). Stage 1 patterns: Facade, Factory Method, Adapter, Command.
Status / change	**Modified.** Generates a summary and cover letter and flags job skills that the resume does not support. No tone option.
Type	Hybrid: the LLM writes the text, the skill check is deterministic.
Classes and responsibilities	`TailoredDocumentService`: asks the LLM for the documents and runs the skill check using `SkillExtractor` and `SourceVerifier`. `TailoredDocument`: summary, cover letter and flagged skills. `TailorDocumentTool` (agent tool), `ApiServer` (`POST /api/documents/tailor`), GUI `/tailor`.
Key methods	`TailoredDocumentService.tailor(resume, job)`, `TailorDocumentTool.execute(args)`
Design patterns	Adapter, Facade (through the agent).
Collaboration	The Tailor page sends the resume and job text to `TailoredDocumentService`. It gets the summary and cover letter from the LLM, extracts the job skills, and flags any that the resume text does not support. The page shows the flags as a warning. In the CLI it is reached through the `agent` and `export` commands.
Video	F04-F10-cover-letter-and-export.mp4 GUI from 0:00, CLI from 0:58

F05 Pipeline State Management

Stage 1 reference	F05 (UC-05, SD-05). Stage 1 patterns: State, Command.
Status / change	**Modified.** Stages renamed (Wishlist to Saved, Interviewing to Interview), moves by buttons, data in memory.
Type	Deterministic.
Classes and responsibilities	`JobApplication`: context; holds its current `ApplicationState`. `ApplicationState` with `SavedState`, `AppliedState`, `InterviewState`, `OfferState`, `RejectedState`: each allows only its legal moves. `InvalidTransitionException`: raised for an illegal move. `ApplicationService`, `ApplicationView`, `TrackCommand` (CLI), `ApiServer` (`/api/applications`), GUI `/pipeline`.
Key methods	`JobApplication.apply()`, `scheduleInterview()`, `receiveOffer()`, `reject()`; `ApplicationService.add(company, title)`, `move(id, action)`, `list(status)`
Design patterns	State, Command (CLI).
Collaboration	A pipeline button or the `track` command calls `ApplicationService.move(id, action)`. The service calls the matching method on `JobApplication`, which delegates to its current state. A legal move returns the next state; an illegal move raises `InvalidTransitionException` (HTTP 409 in the API, an error line in the CLI).
Video	F05-pipeline.mp4 GUI from 0:00, CLI from 0:53

F06 Deadline Tracking & Alerts

Stage 1 reference	F06 (UC-06, SD-06). Stage 1 pattern: Observer (`DeadlineSubject`, `DashboardObserver`, `NotificationAlertObserver`).
Status / change	**Modified.** 7-day window instead of 48 hours; alerts are created when "Check deadlines" is used, not by a background scheduler; the GUI reads unread alerts. A CLI `alerts` command was added (Stage 1 had none for F06).
Type	Deterministic.
Classes and responsibilities	`Deadline`: label and date. `DeadlineScheduler`: finds deadlines inside the window and publishes one `SCHEDULER` alert per deadline. `AlertService`: the subject; stores alerts and notifies observers. `AlertObserver` / `ConsoleAlertObserver`, `Alert`, `AlertSource` (scheduler vs user alerts). `AlertsCommand` (CLI), `ApiServer` (`/api/deadlines`, `/api/alerts`), dashboard page.
Key methods	`DeadlineScheduler.check(deadlines, today)`, `AlertService.subscribe(observer)`, `publish(message, source)`, `unread(source)`, `markRead(id)`, `markAllRead()`
Design patterns	Observer, Command (CLI).
Collaboration	The user adds a deadline on the dashboard. "Check deadlines" calls `POST /api/deadlines/check`, which runs `DeadlineScheduler.check`. For each deadline inside the window (once per deadline) it calls `AlertService.publish`, which stores the `Alert` and notifies subscribed observers. The dashboard lists unread alerts; the `alerts` command lists them and marks them read.
Video	F06-deadlines-alerts.mp4 GUI from 0:00, CLI from 0:36

F07 Interview Prep Sheet

Stage 1 reference	F07 (UC-07, SD-07). Stage 1 patterns: Facade, Factory Method, Adapter, Command.
Status / change	**Implemented as designed** (5 technical and 3 behavioral questions). It also takes the resume as input.
Type	AI-based.
Classes and responsibilities	`PrepSheetService`: builds the prompt through the factory, calls the LLM, parses the JSON into a `PrepSheet`. `PrepSheetPromptFactory` and `PrepSheetPrompt` (with `PromptFactory`, `Prompt`). `PrepSheetTool` (agent tool), `PrepCommand` (CLI), `ApiServer` (`POST /api/prep`), GUI `/prep`.
Key methods	`PrepSheetService.generate(job, resume)`, `PromptFactory.create(inputs)`, `PrepSheet.describe()`
Design patterns	Factory Method, Adapter, Command (CLI).
Collaboration	The Prep page or `prep` command calls `PrepSheetService.generate`. The service asks `PrepSheetPromptFactory.create` for a `PrepSheetPrompt` (the factory checks that the required inputs are present), sends its text to `LLMClient`, and parses the JSON reply into a `PrepSheet`.
Video	F07-prep-sheet.mp4 GUI from 0:00, CLI from 0:27

F08 Bullet Point Optimizer

Stage 1 reference	F08 (UC-08, SD-08). Stage 1 patterns: Facade, Factory Method, Adapter, Command.
Status / change	**Implemented as designed** (three rewrites), plus a check for numbers that were not in the original bullet.
Type	Hybrid: the LLM rewrites, the figure check is deterministic.
Classes and responsibilities	`BulletOptimizerService`: builds the prompt through the factory, calls the LLM, checks each version. `BulletPromptFactory` and `BulletOptimizerPrompt`. `BulletResult` and `BulletVersion(text, unsupportedFigures)`. `OptimizeBulletTool` (agent tool), `BulletCommand` (CLI), `ApiServer` (`POST /api/bullets/optimize`), GUI `/bullets`.
Key methods	`BulletOptimizerService.optimize(bullet, job)`, `BulletResult.describe()`
Design patterns	Factory Method, Adapter, Command (CLI).
Collaboration	The Bullets page or `bullet` command calls `optimize`. The service gets a `BulletOptimizerPrompt` from `BulletPromptFactory`, calls the LLM, parses three versions, and lists in each `BulletVersion` any figures that do not appear in the original bullet.
Video	F08-bullet-optimizer.mp4 GUI from 0:00, CLI from 0:30

F09 Analytics Dashboard

Stage 1 reference	F09 (UC-09, SD-09). Stage 1 pattern: Command (CLI only).
Status / change	**Modified.** Counts and rates are computed in memory and shown as numbers in a panel on the pipeline page, not as charts.
Type	Deterministic.
Classes and responsibilities	`MetricsService`: reads the applications and computes counts per stage and interview, offer and rejection rates. `DashboardMetrics`: the result record. `StatsCommand` (CLI), `ApiServer` (`GET /api/analytics`), analytics panel on `/pipeline`.
Key methods	`MetricsService.compute()`
Design patterns	Command (CLI).
Collaboration	The pipeline page or `stats` command asks `MetricsService.compute()`, which reads the list from `ApplicationService` and returns a `DashboardMetrics` record for display.
Video	F09-analytics.mp4 GUI from 0:00, CLI from 0:30

F10 Application Package Exporter

Stage 1 reference	F10 (UC-10, SD-10). Stage 1 pattern: Command (CLI only).
Status / change	**Modified.** Exports one plain text file with a review warning on top, instead of a PDF.
Type	Deterministic.
Classes and responsibilities	`ExportService`: assembles summary, cover letter and resume into text and saves it under `exports/` (git-ignored). `ExportFile(filename, content)`. `ExportCommand` (CLI), `ApiServer` (`POST /api/export`), Export button on `/tailor`.
Key methods	`ExportService.build(label, resume, summary, coverLetter, ...)`, `ExportService.save(file, directory)`
Design patterns	Command (CLI).
Collaboration	After a tailored document is generated, the Export button (or the `export` command) calls `ExportService.build`, which formats the text and puts a warning at the top when skills were flagged. `save` writes the file to the export folder.
Video	F04-F10-cover-letter-and-export.mp4 GUI from 0:00, CLI from 0:58

F11 Career Agent Assistant

Stage 1 reference	F11 (UC-11, SD-11). Stage 1 classes: `Planner`, `ToolManager`, `MemoryManager`, `ResponseValidator`, `AIEngineFacade`.
Status / change	**Modified.** One tool per request (Figure 9); `ToolManager` became `ToolRegistry` with five tools; no memory, validator class, replanning or clarifying question. See Section 4.
Type	Hybrid: the LLM chooses the tool and writes the final answer, deterministic code validates the choice and runs the tool.
Classes and responsibilities	`AgentController`: single entry point; coordinates planning, tool execution and the final answer. `Planner`: asks the LLM for a JSON decision and validates it; `PlanStep` holds the result. `ToolRegistry`: registers and finds tools. `Tool`: interface for the five tools `ExtractSkillsTool`, `MatchJobTool`, `TailorDocumentTool`, `OptimizeBulletTool`, `PrepSheetTool`. `PlanningException`, `AgentCommand` and `AskCommand` (CLI), `ApiServer` (`POST /api/agent/ask`), GUI `/agent`.
Key methods	`AgentController.handle(request)`, `Planner.plan(request)`, `ToolRegistry.register(tool)`, `find(name)`, `describeAll()`, `Tool.execute(args)`
Design patterns	Facade (`AgentController`), Adapter, Command (CLI).
Collaboration	See Figure 9. `handle` asks `Planner.plan`, which sends the tool list and the request to the LLM and checks that the reply is valid JSON naming a registered tool (or "none" with an answer). A direct answer is returned as is. Otherwise the controller runs the tool and makes a second LLM call to write a short final answer using only the tool result. Planning, argument and tool failures come back as readable messages.
Video	F11-career-agent.mp4 GUI from 0:00, CLI from 0:24

F12 Mock Interview Practice

Stage 1 reference	F12 (UC-12, SD-12). Stage 1 patterns: Facade, Factory Method, Adapter, Command.
Status / change	**Implemented as designed.** Sessions live in memory. Answers are also checked for skill claims that are not in the resume.
Type	Hybrid: the LLM writes questions and feedback, the claim check is deterministic.
Classes and responsibilities	`MockInterviewService`: starts a session, evaluates each answer, finishes with a summary. `InterviewSession`: questions, status and feedback of one session. `InterviewQuestion`, `AnswerFeedback`, `InterviewSummary`, `InterviewException` / `InterviewNotFoundException`. `InterviewCommand` (CLI), `ApiServer` (`/api/interview/...`), GUI `/interview`.
Key methods	`MockInterviewService.start(resume, job, count)`, `answer(sessionId, questionId, answer)`, `finish(sessionId)`, `get(sessionId)`
Design patterns	Adapter, Command (CLI).
Collaboration	`start` asks the LLM for questions and creates an `InterviewSession`. Each `answer` call sends the question and answer to the LLM for a score and feedback, then flags skills the answer claims that the resume does not support, and stores an `AnswerFeedback`. `finish` returns an `InterviewSummary` with the average score.
Video	F12-mock-interview.mp4 GUI from 0:00, CLI from 1:11

Additions beyond Stage 1

ID	Behavior and reason	Classes and methods	Stage 1 link and patterns	Video
N1	Every skill the LLM extracts must appear in the source text; unsupported skills are reported as rejected. Added so the AI cannot invent skills.	`SourceVerifier.isSupported`, `ExtractionResult.rejected`, `SkillExtractor.extract`	New. Supports UC-01 (F01).	Shown in the F01 video.
N2	Factual checks on AI output: skills flagged on tailored documents, skill claims flagged on interview answers, unsupported figures flagged on bullets. Answers the Stage 1 feedback to keep factual-source checks.	`TailoredDocumentService.tailor`, `MockInterviewService.answer`, `BulletVersion.unsupportedFigures`	New. Supports UC-04, UC-08, UC-12.	Shown in the F04, F08 and F12 videos.
N3	Reliability layer for the LLM: retry with backoff on 429 and 5xx, readable error messages, one extra attempt when skill extraction returns no JSON, and a `MockLLMClient` for deterministic tests.	`GeminiClient.generate`, `SkillExtractor.extract`, `MockLLMClient`	New. Adapter pattern (Figure 8).	Not a user feature; exercised by every demo.
N4	Javalin REST layer and `AppContext`, so the GUI and the CLI use one set of services.	`ApiServer.create`, `AppContext.load`, `AppContext.withLlm`	Replaces the Stage 1 REST controllers and repositories.	Not a user feature; exercised by every demo.

3. Implemented design patterns

Seven distinct patterns are implemented. Figures 2 to 8 show the participating classes (all UMLet; sources are in `stage2/diagrams/umlet/`), Figure 1 shows the whole system, and Figure 9 redraws the agent sequence diagram. Stage 1 listed the same seven patterns, but several roles changed; each pattern below states whether it matches Stage 1.

3.1 State: application pipeline (Figure 2)

Design problem	An application may only move through legal stages (Saved, Applied, Interview, Offer, Rejected). Checking that with conditions scattered across the code is error-prone.
Participants and roles	`JobApplication` (context), `ApplicationState` (state interface), `SavedState`, `AppliedState`, `InterviewState`, `OfferState`, `RejectedState` (concrete states), `InvalidTransitionException`, `ApplicationService` (client).
Source files and methods	`pipeline/*.java`: `JobApplication.apply()`, `scheduleInterview()`, `receiveOffer()`, `reject()`; `ApplicationState` default methods; `ApplicationService.move(id, action)`.
How it follows the pattern	`JobApplication` keeps a reference to its current state and forwards each action to it. Each state implements only its legal actions and returns the next state; any other action raises `InvalidTransitionException`. Adding a stage means adding a class, not editing conditionals.
Matches Stage 1?	**Yes, with renames.** `WishlistState` became `SavedState`, `InterviewingState` became `InterviewState`, and `handleTransition`/`changeState` became one method per action.

3.2 Strategy: job matching (Figure 3)

Design problem	A skill match can be computed by a fast deterministic rule or by asking the LLM. The caller should not depend on which.
Participants and roles	`MatchingStrategy` (strategy interface), `KeywordOverlapStrategy` and `LlmMatchingStrategy` (concrete strategies), `JobMatcher` (context), `MatchResult`.
Source files and methods	`matching/*.java`: `JobMatcher.setStrategy(strategy)`, `JobMatcher.match(resumeSkills, jobSkills)`, `MatchingStrategy.match(...)`.
How it follows the pattern	`JobMatcher` holds a `MatchingStrategy` and delegates to it; the strategy can be swapped at run time with `setStrategy`. `AppContext` starts with the keyword strategy, which needs no network and is easy to test.
Matches Stage 1?	**No, the use changed.** Stage 1 used Strategy for input extraction (`ExtractionStrategy` with `PDFResumeExtractor`, `WebJobScraper`, `PastedTextJobParser`). In the implementation ingestion has only two input forms, handled by `JobIngestionService` and a `PageFetcher` interface, so a Strategy there would have added classes without removing a real conditional. Matching has two genuinely interchangeable algorithms, so Strategy is applied there instead. Impact: F02 no longer uses Strategy; F03 does.

3.3 Observer: alerts (Figure 4)

Design problem	Several consumers (CLI output, other displays) need to hear about new alerts without the producer knowing them.
Participants and roles	`AlertService` (subject), `AlertObserver` (observer interface), `ConsoleAlertObserver` (concrete observer), `DeadlineScheduler` (event producer), `Alert`, `AlertSource`, `Deadline`.
Source files and methods	`alerts/*.java`: `AlertService.subscribe(observer)`, `unsubscribe(observer)`, `publish(message, source)`, `DeadlineScheduler.check(deadlines, today)`.
How it follows the pattern	`publish` stores the alert and calls `onAlert` on every subscribed observer. `DeadlineScheduler` and user actions both publish through the same subject; `AlertSource` keeps scheduler alerts distinct from user alerts, as the Stage 1 feedback asked.
Matches Stage 1?	**Partly.** Stage 1 had `DeadlineSubject`, `DashboardObserver` and `NotificationAlertObserver`, a 48-hour window and alerts pushed to the GUI. The implementation uses `AlertService` as the subject and `ConsoleAlertObserver` for the CLI, a 7-day window, and a manual check. The GUI reads unread alerts through the REST API because the API has no push channel. The subject/observer structure is kept.

3.4 Command: CLI (Figure 5)

Design problem	The CLI must turn typed text into actions without one large if/else block.
Participants and roles	`CommandRegistry` (invoker), `Command` (command interface), twelve concrete commands: `HelpCommand`, `AskCommand`, `AgentCommand`, `AlertsCommand`, `TrackCommand`, `StatsCommand`, `BulletCommand`, `PrepCommand`, `ExportCommand`, `InterviewCommand`, `ResumeCommand`, `JobCommand`.
Source files and methods	`cli/*.java`: `CommandRegistry.register(command)`, `dispatch(line)`, `Command.name()`, `description()`, `execute(args)`.
How it follows the pattern	`dispatch` splits the line, finds the command registered under that name and calls `execute`. Each command wraps one request and calls the shared services; adding a command means adding a class and one `register` call.
Matches Stage 1?	**Partly.** Stage 1 had `CommandInvoker` and eleven commands (for example `ImportResumeCommand`, `AnalyzeMatchCommand`) that called REST controllers. The implementation has `CommandRegistry` and twelve commands that call the services directly in the same process. Match analysis and cover letters are reached through `agent` and `export` instead of their own commands.

3.5 Factory Method: prompts (Figure 6)

Design problem	Prompts need required inputs and a fixed structure; building them inline in services duplicates text and hides missing inputs.
Participants and roles	`PromptFactory` (abstract creator), `BulletPromptFactory` and `PrepSheetPromptFactory` (concrete creators), `Prompt` (product interface), `BulletOptimizerPrompt` and `PrepSheetPrompt` (concrete products).
Source files and methods	`prompts/*.java`: `PromptFactory.create(inputs)` (final), `createPrompt(inputs)` and `requiredInputs()` (abstract), `Prompt.text()`.
How it follows the pattern	`create` checks `requiredInputs()` and then calls the subclass factory method `createPrompt`, which returns the right `Prompt`. The services use only the `Prompt` interface.
Matches Stage 1?	**Partly.** Stage 1 listed seven concrete factories (match, cover letter, prep sheet, bullet, resume parse, planning, mock interview). Only the bullet and prep sheet factories were built. The other prompts are assembled inside their services or in `Planner`. This is an inconsistency listed in Section 1.

3.6 Facade: agent (Figure 7)

Design problem	The CLI and the API need one simple call to run an agent request without knowing about planning, tool lookup, execution and answer writing.
Participants and roles	`AgentController` (facade); `Planner`, `ToolRegistry`, `Tool` and its five implementations (subsystem); `PlanStep`, `PlanningException`.
Source files and methods	`agent/*.java`: `AgentController.handle(request)`, `Planner.plan(request)`, `ToolRegistry.find(name)`, `Tool.execute(args)`.
How it follows the pattern	`handle` is the only method callers use. It hides the order of planning, validation, tool execution and the second LLM call, and it turns every failure into a readable message.
Matches Stage 1?	**No, a different facade.** Stage 1 had `AIEngineFacade` over `PromptFactory`, `LLMClient`, `JSONResponseParser` and `ResponseValidator`. It was not built: each service parses its own JSON and calls `LLMClient` directly, so a shared facade would have been a pass-through. The Facade role is played by `AgentController` for the agent subsystem.

3.7 Adapter: LLM access (Figure 8)

Design problem	The application should not depend on Gemini's request and response format, and tests must run without a network.
Participants and roles	`LLMClient` (target interface), `GeminiClient` (adapter), the Gemini REST API (adaptee, reached through the nested `Transport` interface using Java's `HttpClient`), `MockLLMClient` (test double).
Source files and methods	`llm/*.java`: `LLMClient.generate(prompt)`, `GeminiClient.generate`, `buildRequestBody`, `parseResponse`, `describeError`.
How it follows the pattern	`GeminiClient` implements `LLMClient`, translates a prompt into the Gemini JSON request, retries 429 and 5xx responses up to three times, and converts the reply and errors into a string or `LLMException`. All services depend only on `LLMClient`.
Matches Stage 1?	**Yes.** Stage 1 named `GeminiClient` as the adapter and `GeminiRestAPI` as the adaptee; the adaptee is the external REST endpoint reached through `Transport`, not a separate class. Retry and `MockLLMClient` were added.

Diagrams updated since Stage 1

The system class diagram (Figure 1) is new. Use cases UC-01 to UC-12 are unchanged: every use case is implemented, although some alternative flows differ as described in Section 4. Of the Stage 1 sequence diagrams, SD-11 changed most (no `MemoryManager`, `ToolManager`, `ResponseValidator`, replanning or clarifying question), so it is redrawn as Figure 9. SD-03, SD-04, SD-07, SD-08 and SD-12 pass through `AIEngineFacade`, which was not built: in the implementation the service calls the prompt factory (where one exists) and `LLMClient` directly. SD-06 uses `DeadlineSubject` and `DashboardObserver` (now `AlertService` and a pulled alert list), SD-10 produces a PDF (now plain text) and SD-13 calls REST controllers (now the services). The message order of those diagrams is otherwise the same.

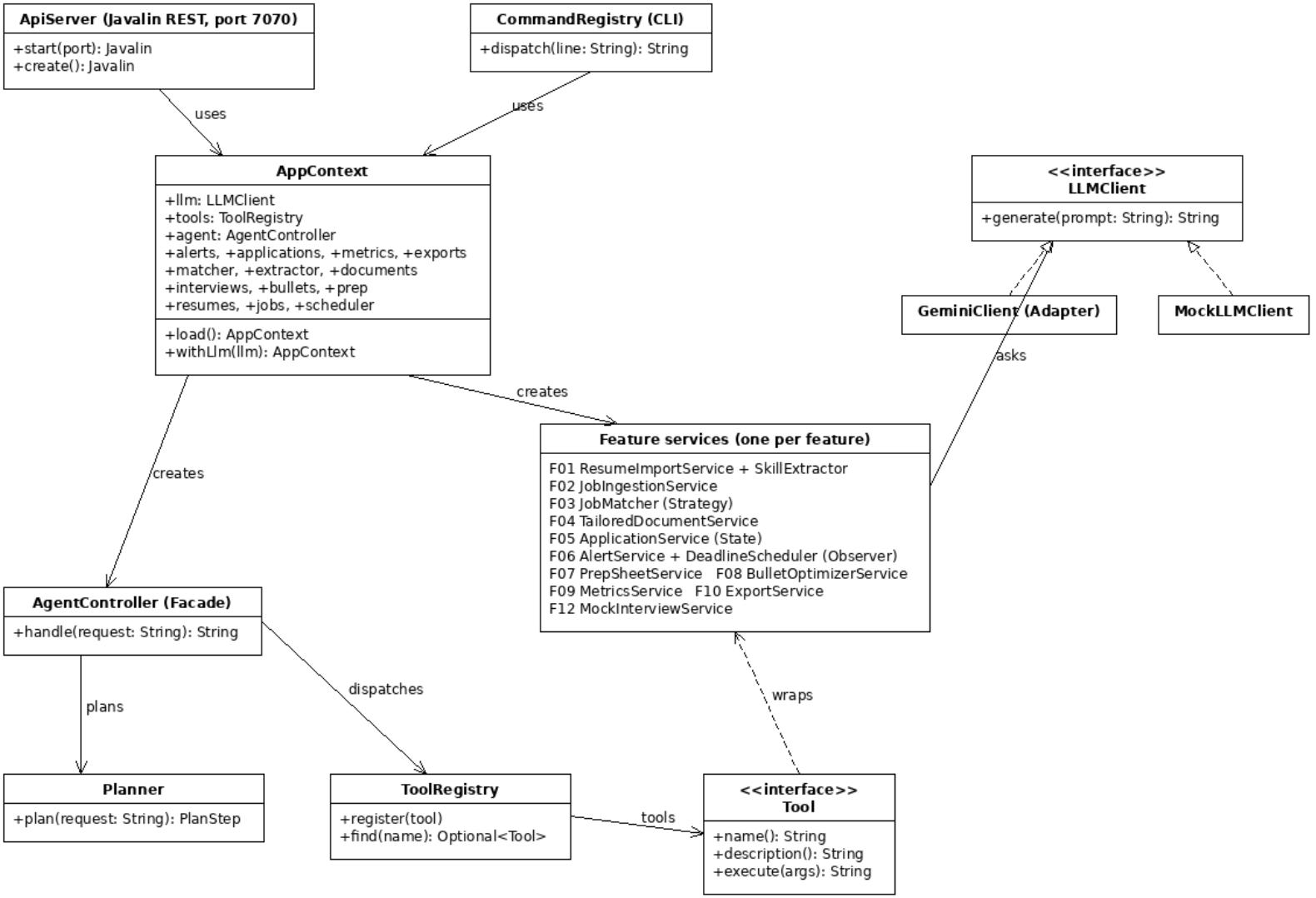


Figure 1. System overview: interfaces, AppContext, services, agent and LLM access.

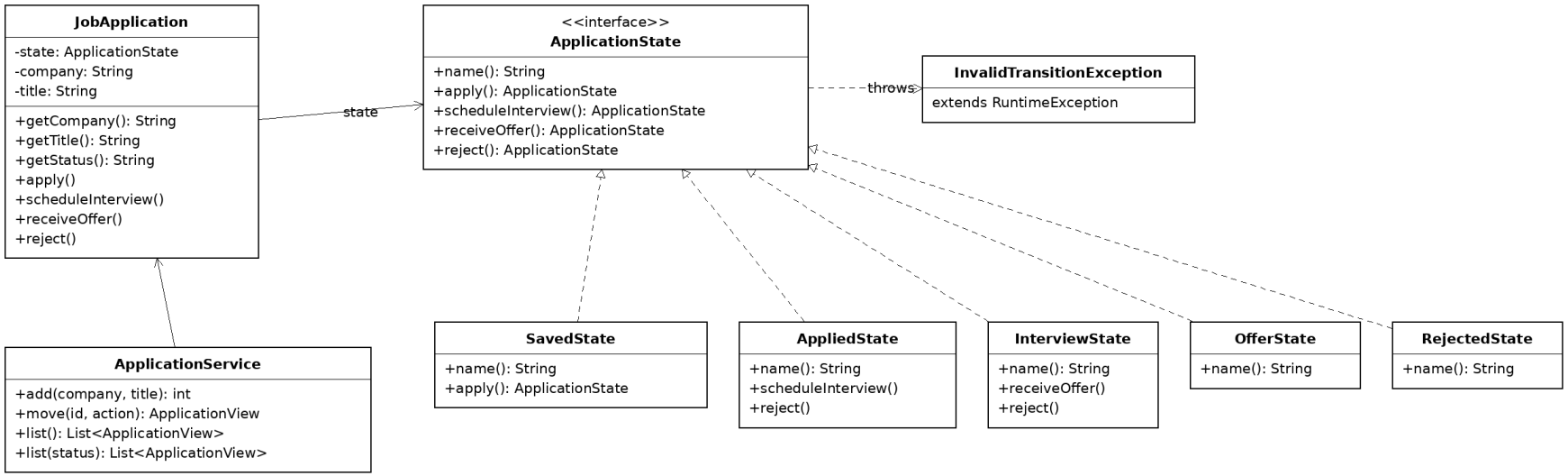


Figure 2. State pattern: application pipeline (package pipeline).

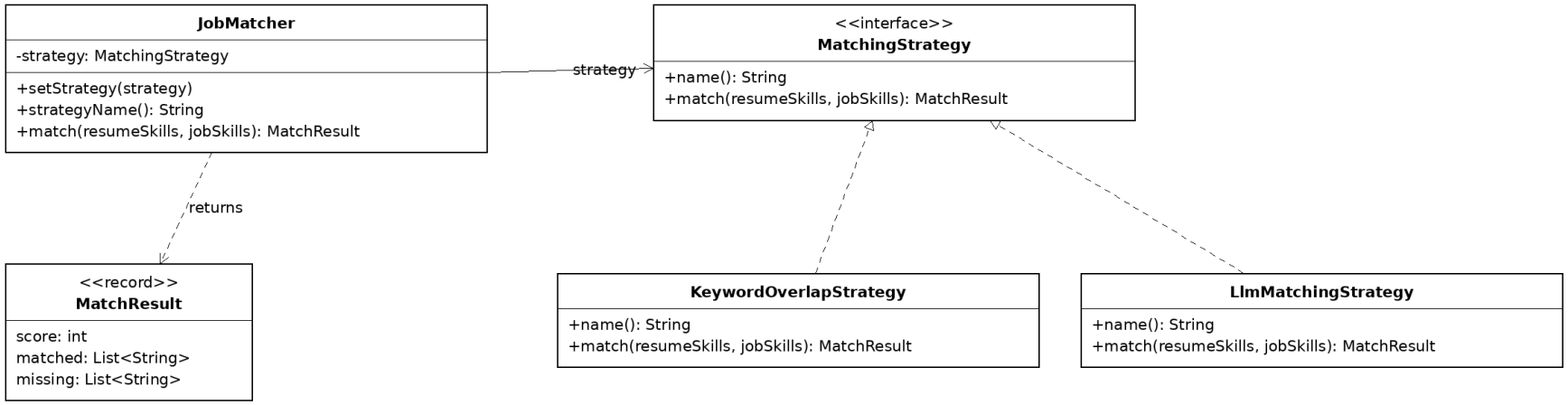


Figure 3. Strategy pattern: job matching (package matching).

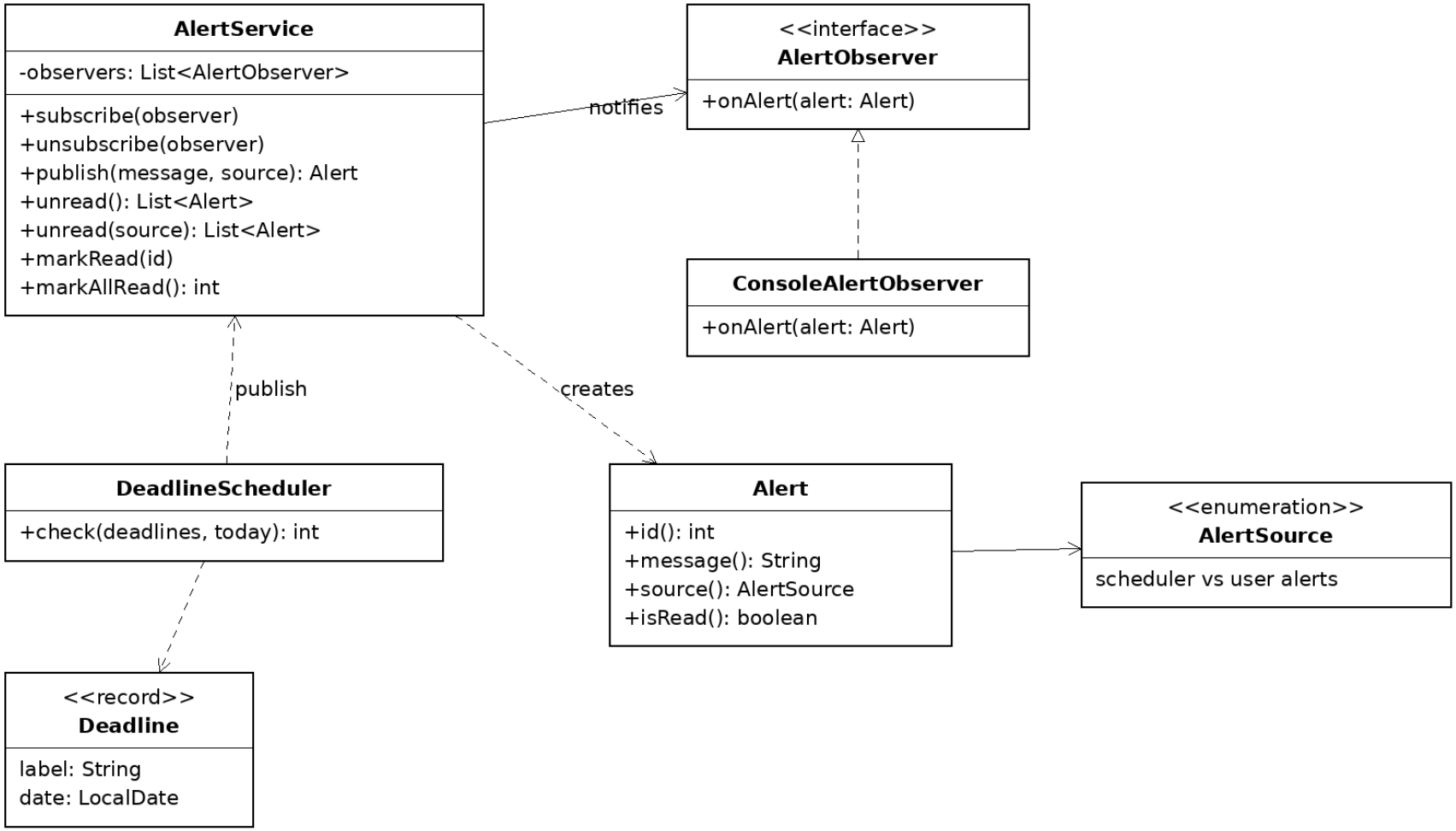


Figure 4. Observer pattern: deadline alerts (package alerts).

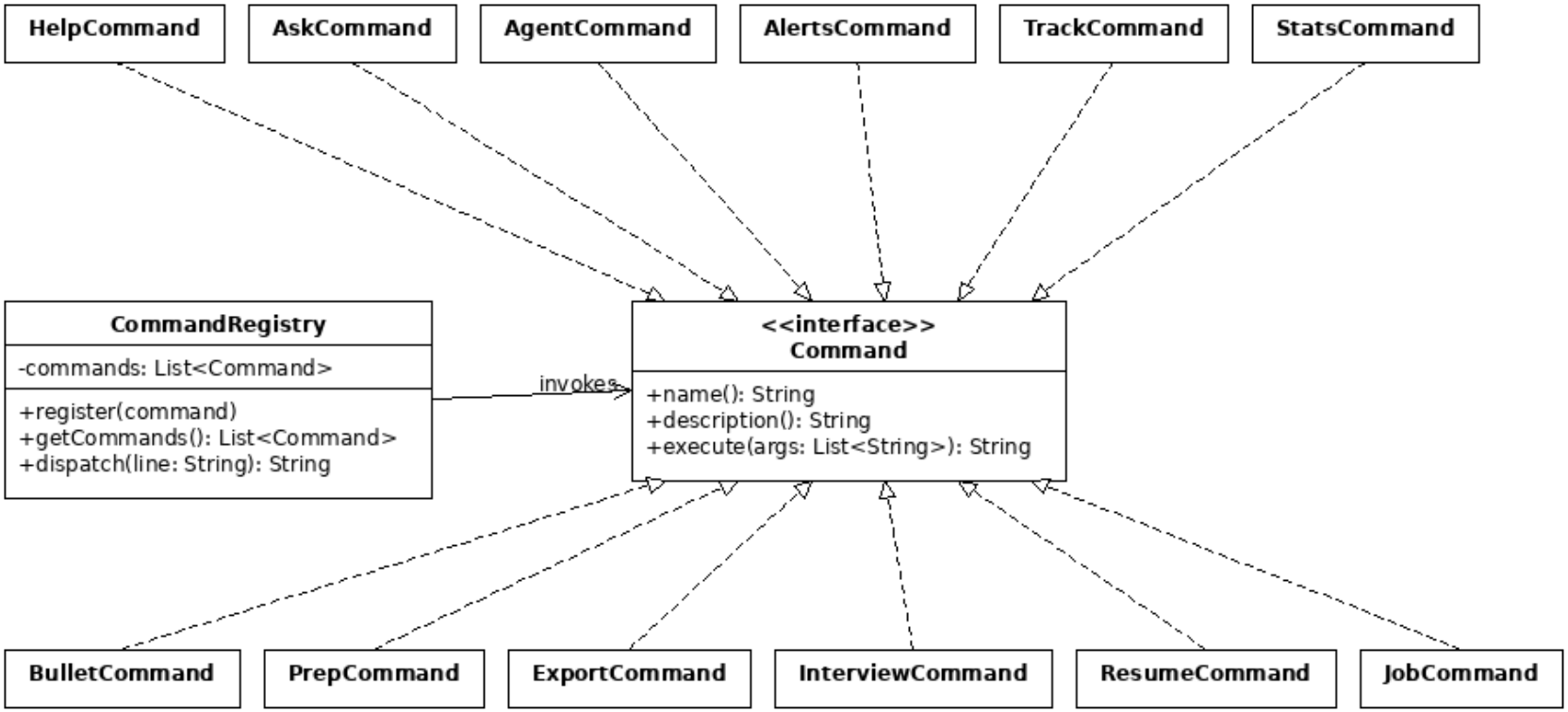


Figure 5. Command pattern: CLI commands (package cli).

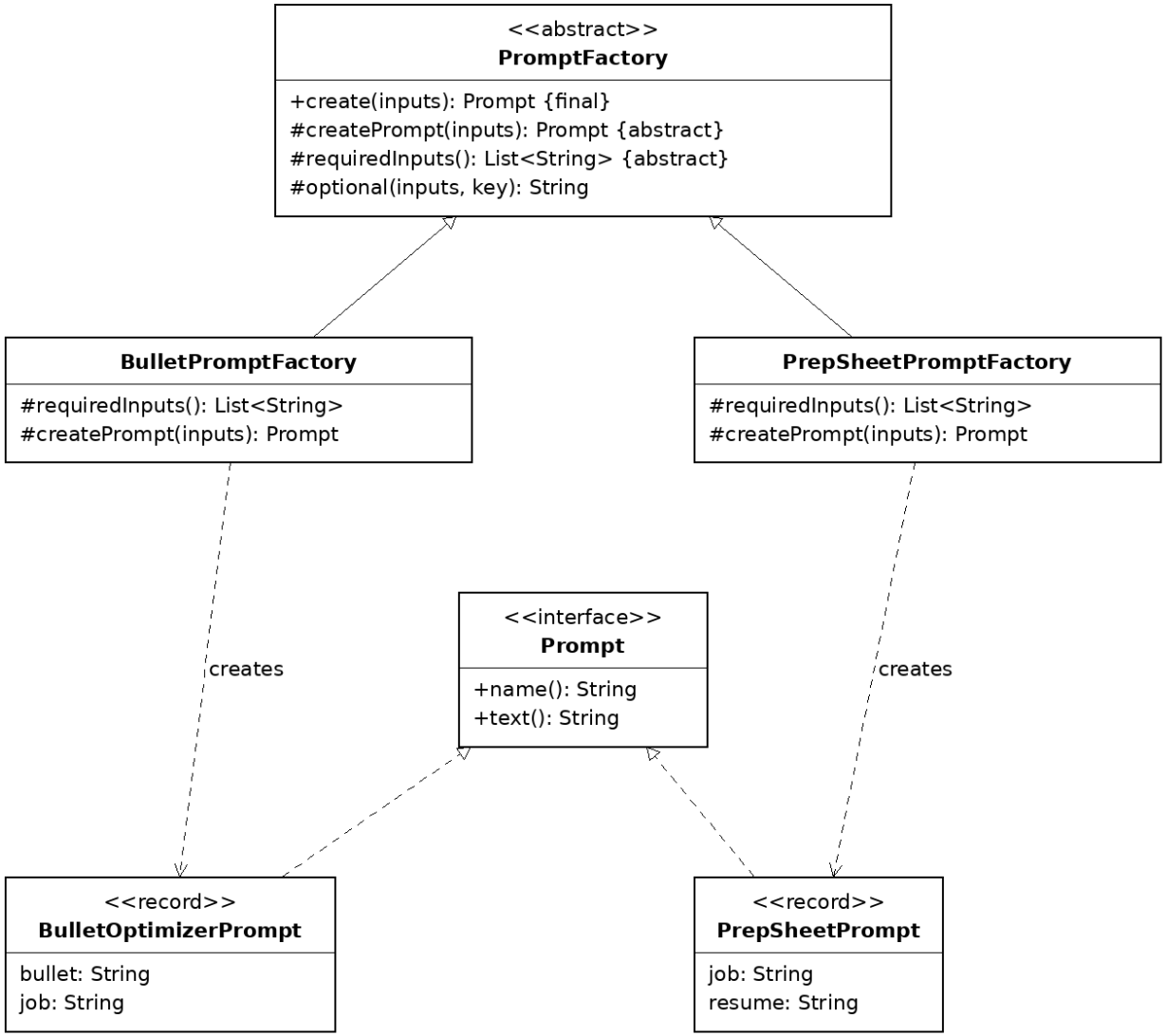


Figure 6. Factory Method pattern: prompt factories (package prompts).

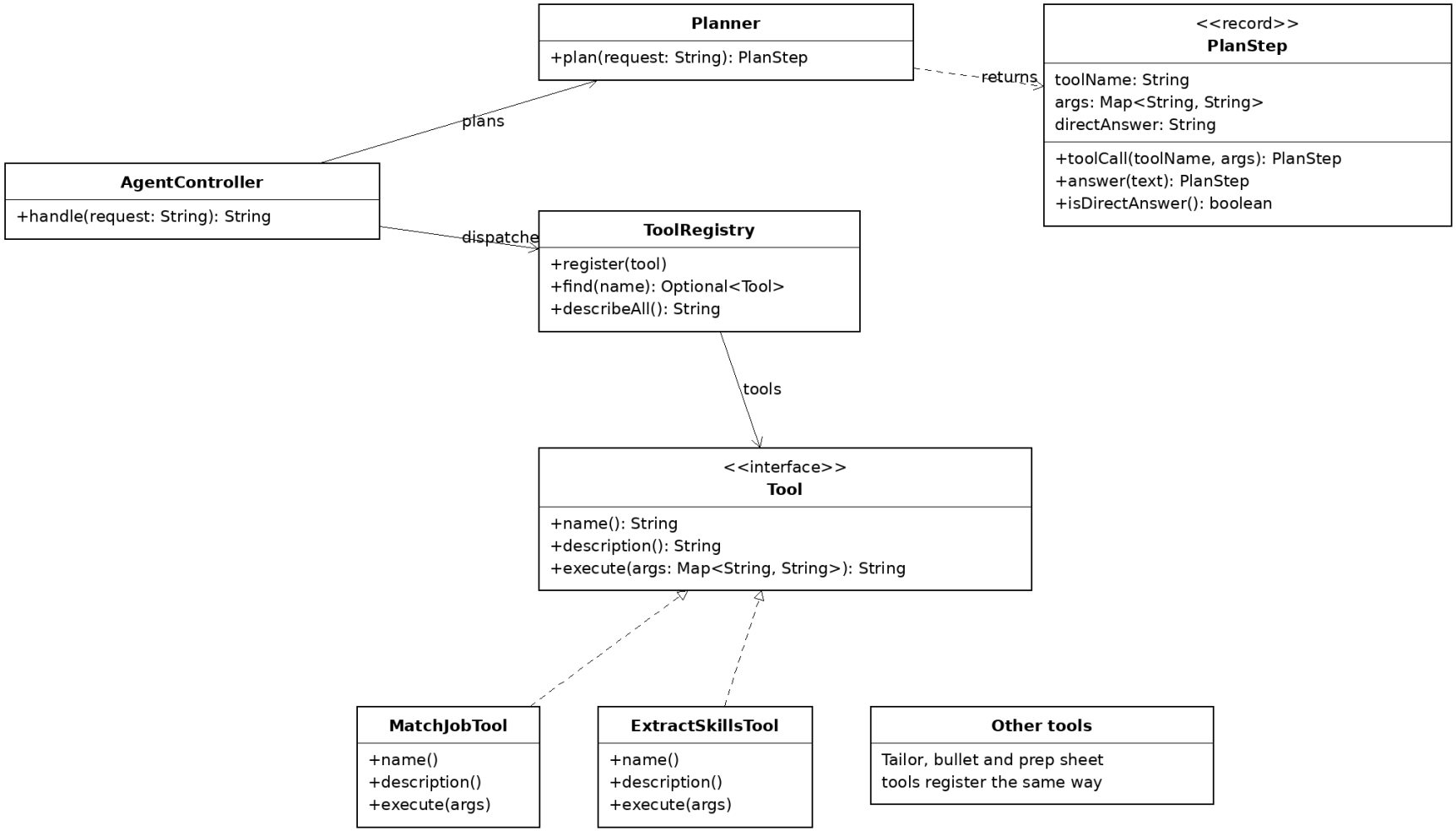


Figure 7. Facade: agent controller over the planner and tool registry (package agent).

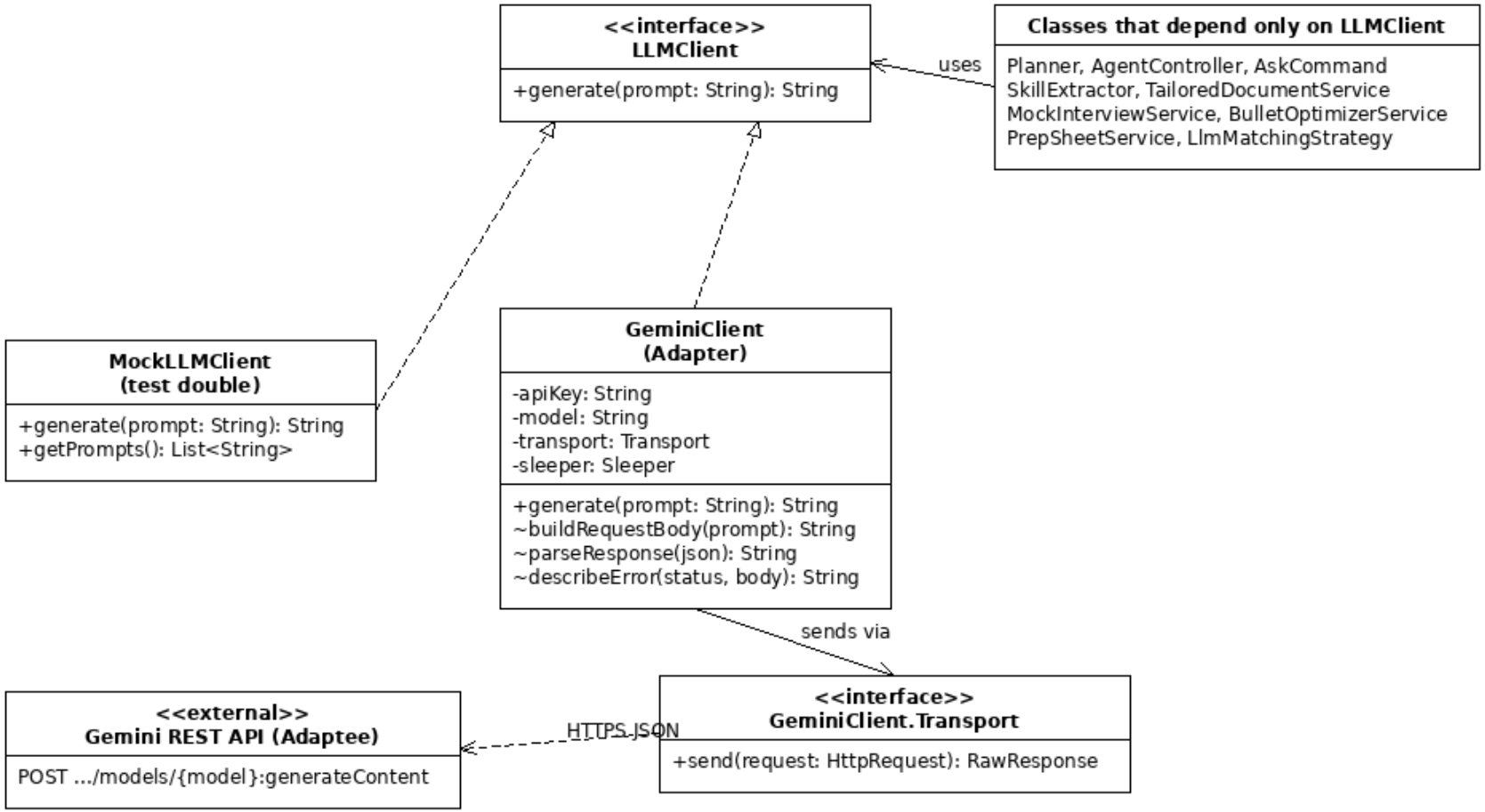


Figure 8. Adapter pattern: LLM access (package llm).

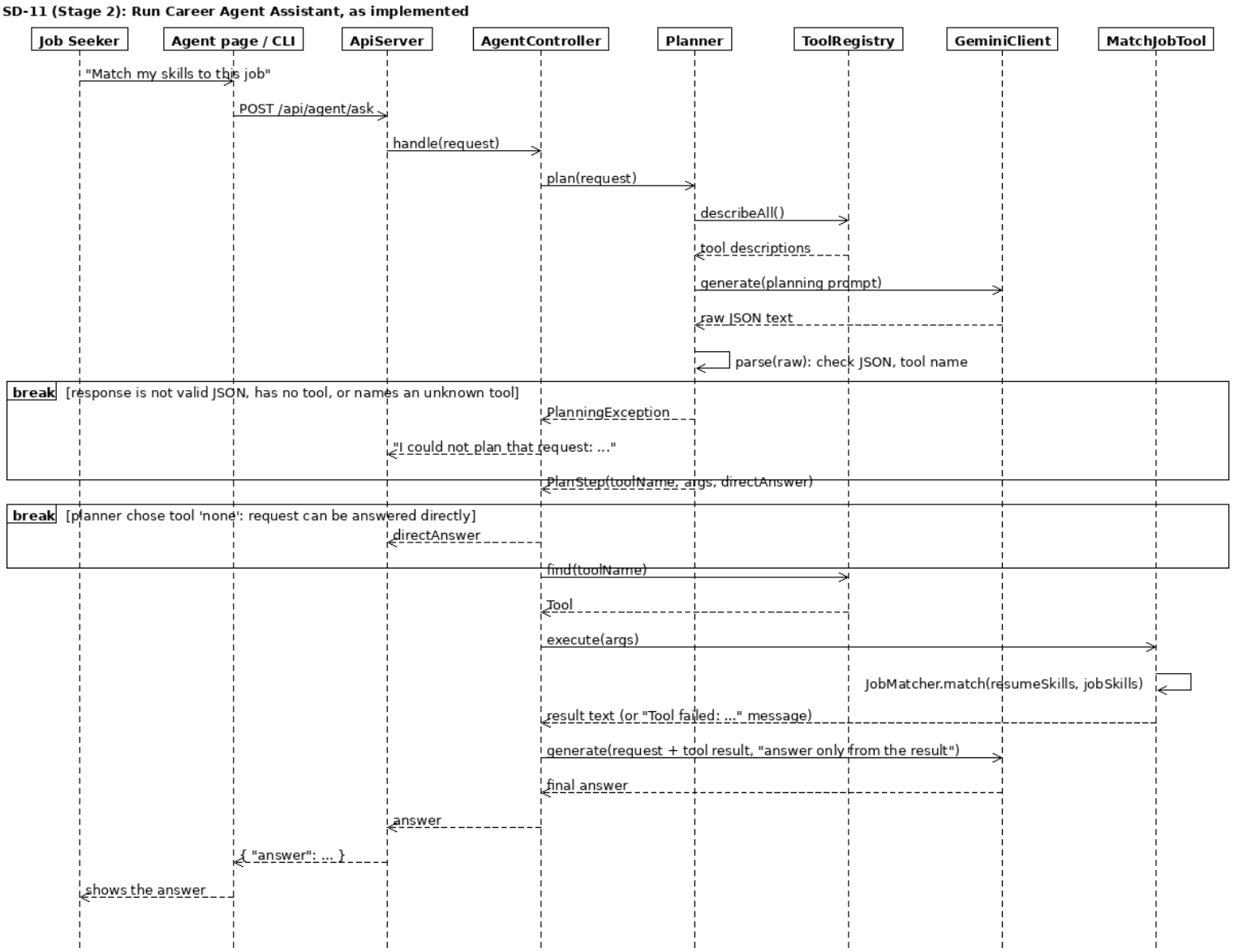


Figure 9. Sequence diagram SD-11 as implemented: run the career agent assistant.

4. Changes and rationale

Each row follows Original design, Implemented change, Reason, Impact. Section 2 lists the additions (N1 to N4) and Section 3 the pattern changes in more detail.

Feature changes

Item	Original design (Stage 1)	Implemented change	Reason	Impact
F01	Skills, experience and education from a PDF, saved to PostgreSQL.	Skills only, from a PDF or pasted text, kept in memory.	Applicant tracking systems scan mainly for skills and keywords, and skills can be checked against the resume text.	No experience or education fields.
F02	Strategy-based scraper and parser; result shown as a Kanban card.	URL (http/https only, private addresses refused) or pasted text, cleaned to plain text and listed on the Jobs page.	Only two input forms; a branch and a `PageFetcher` interface are enough. The address check was added for safety.	No Kanban card. Blocked sites need pasted text.
F03	LLM compares resume and job semantically.	Strategy with keyword overlap (default) and an LLM strategy.	A deterministic score is testable and still works when the LLM is rate-limited.	Default score is exact skill overlap, not semantic.
F04	Cover letter with optional tone.	Summary and cover letter, with unsupported job skills flagged. No tone.	Stage 1 feedback asked for factual-source checks; this was prioritised over tone.	No tone input.
F05	Wishlist/Interviewing states, drag-and-drop Kanban, PostgreSQL.	Saved/Interview states, buttons, in-memory.	Clearer names; legal moves were tested first and drag-and-drop was left out.	Button UI; data resets on restart.
F06	Background scheduler, 48-hour window, alerts pushed to the GUI.	7-day window; "Check deadlines" runs the check; GUI reads unread alerts; CLI `alerts` added.	The REST API has no push channel, weekly planning suits a 7-day window, and Stage 1 feedback asked for scheduler alerts to stay distinct (`AlertSource`) and unread deadlines to reach the CLI.	Alerts are not automatic.
F07, F08, F12	Prep sheet, three bullet rewrites, mock interview.	As designed. Prep sheet also takes the resume; bullets and interview answers get factual checks.	Stage 1 feedback on factual checks.	Extra input and flags.
F09	SQL aggregation and charts.	In-memory counts and rates shown as numbers.	No database; numbers were enough to verify the logic.	No charts.
F10	PDF with plain-text fallback.	Plain text with a review warning on top.	Deterministic, no document library, and the warning stays in the file.	No PDF output.
F11	Multi-step `Plan`, `ToolManager` with six tools, `MemoryManager`, `ResponseValidator`, replanning, clarifying question.	One tool per request; `ToolRegistry` with five tools; validation in `Planner.parse` and `ToolRegistry.find`; a second LLM call writes the final answer; failures return	A small agent that could be tested end to end first.	No multi-step plans, memory, replanning or clarification.

Item	Original design (Stage 1)	Implemented change	Reason	Impact
		messages.		

Architecture, class and pattern changes

Item	Original design (Stage 1)	Implemented change	Reason	Impact
Persistence	PostgreSQL and repositories.	In-memory services.	All twelve features were prioritised over adding a database.	Data resets on restart.
AI engine	`AIEngineFacade`, `JSONResponseParser`, `ResponseValidator`.	Not built; services call `LLMClient` directly.	Each feature parses its own JSON, so a shared facade would only pass calls through.	Facade pattern moved to `AgentController`.
Strategy	Input extraction strategies.	Matching strategies.	See 3.2.	F02 no longer uses Strategy.
Factory Method	Seven prompt factories.	Two (bullet, prep sheet).	Built where first needed; not extended.	Other prompts live in services.
Observer	`DeadlineSubject`, dashboard and notification observers.	`AlertService`, `ConsoleAlertObserver`; GUI pulls.	See 3.3.	No real-time GUI updates.
Command	`CommandInvoker`, eleven commands calling REST controllers.	`CommandRegistry`, twelve commands calling services.	A registry keyed by command name is simpler to extend and test; same JVM, no HTTP hop.	CLI needs no running server.
Interface layer	REST controllers, Kanban dashboard.	`ApiServer` (Javalin) plus ten Next.js pages and `AppContext`.	One set of services for GUI and CLI.	Single composition root.
LLM	Gemini 3.1 Pro.	gemini-3.5-flash-lite through `LLM_MODEL`; retry with backoff.	Live tests returned HTTP 429 on the 3.1 Pro preview and 503 on another flash model; only flash-lite answered reliably.	Model is set in `.env`.
Alert window	Within 48 hours.	7 days.	Planning is done weekly.	Earlier notice.
Factual check	Source checks on generated text.	Skills, claimed skills and figures only.	These can be verified deterministically against the resume.	Employers and degrees are not checked.

5. AI tools and collaboration log

How AI was used

- **Development:** Claude in a chat workflow generated Java and TypeScript code as PowerShell scripts that create files and make anchored edits. Each change was followed by `mvn test` and a manual check in the browser or CLI before committing.
- **Design:** Gemini drafted the Stage 1 design document; Claude later revised the architecture and diagrams. The Stage 2 UMLet diagrams (Figures 1 and 5 to 9 and the updated sources) were generated with a script and exported as PNG.
- **Runtime:** the Gemini API powers skill extraction, tailored documents, bullet optimization, prep sheets, mock interviews and agent planning.

- **Human review:** the author ran the code, checked scripts against the real files, reset a wrong attempt with git, and decided scope changes (Section 4).

Tool	Model	Used for	Plan and usage	Cost
Claude (claude.ai chat)	Chosen by claude.ai; the exact model was not recorded for every session.	Code generation, debugging, Stage 1 revision, UML diagrams, this report.	Free plan since Oct 26, 2026. Token counts are not shown by the chat interface; long sessions from Oct 26 to Nov 10.	$0
Gemini (Pro subscription)	Gemini Pro (model name not shown by the app).	Stage 1 design drafting.	Free student plan, about 13 months in total. Token counts not shown.	$0
Gemini API	gemini-3.5-flash-lite	All runtime AI features and live testing.	Free tier. The AI Studio usage page (28-day view) shows requests in the tens per day, peaking at about 43 in one day, plus a few 429 and 503 errors.	$0

Token counts are not shown by the chat tools, so their usage is described instead of counted. The Gemini API figures come from the Google AI Studio usage page.

Representative interactions

1. Choosing a working model and handling API errors

Task	Make skill extraction work against the real Gemini API.
Input	Live runs returned HTTP 429 on the 3.1 Pro preview and 503 on another flash model.
AI contribution	Proposed retry with backoff and readable error messages in `GeminiClient`, with unit tests that script responses through `Transport`.
Human contribution	Tried each model by hand, chose gemini-3.5-flash-lite, and moved the model name into `.env`.
Outcome	Retry client (commit bd97f2a); model change recorded in Section 4.

2. Non-JSON replies from the model

Task	Stop skill extraction failing when the model answers without a JSON array.
Input	Intermittent failures seen in the GUI.
AI contribution	Added one retry in `SkillExtractor` and tests for the empty-reply case.
Human contribution	Reproduced the failure, reviewed the retry limit, ran `mvn test`.
Outcome	Commit a6edbda.

3. A failed script and a git reset

Task	Add F09 analytics and F10 export.
Input	The PowerShell scripts did not apply cleanly and the working tree was in doubt.
AI contribution	Diagnosed that F09 was already committed and supplied a corrected F10 script.
Human contribution	Checked `git status` and the diff, reset the failed attempt, applied the corrected script and ran the tests.
Outcome	F10 committed in 6358598 with its tests passing.

4. Unsupported skills in AI output

Task	Keep generated text honest about the resume.
Input	Stage 1 feedback asked to keep factual-source checks.
AI contribution	Suggested a deterministic `SourceVerifier` and applying the same idea to documents, bullets and interview answers.
Human contribution	Chose to limit the checks to skills and figures, which can be verified, and documented the limit.
Outcome	N1 and N2 in Section 2.

5. Unstyled GUI

Task	Get the Next.js pages styled.
Input	Pages loaded without Tailwind styles.
AI contribution	Identified the missing PostCSS setup and supplied the fix.
Human contribution	Confirmed the fix in the browser.
Outcome	Commit 05b9b1d.

6. Development and verification summary

Weekly commit summary

Each commit hash links to its page in the public repository. The course requires at least 5 meaningful code commits in each window. A commit is counted only if it changes at least one source file (java, ts, tsx, js or css); commits that change none are listed but not counted.

Window	All commits	Code commits	Focus
Window 1: Oct 20 to 26	5	5	Backend skeleton, configuration, LLM adapter, State pipeline, Command CLI (all on Oct 26).
Window 2: Oct 27 to Nov 2	7	7	Agent core, skill extraction, Strategy matching, Observer alerts, tracking, Gemini retry.
Window 3: Nov 3 to 9	20	17	REST API, tailored documents, mock interview, GUI, analytics, bullets and prep, exporter, resume and job import. Commits b2b9b05, 0f78a83 and 45b6615 change no source files and are not counted.

Window 1 (Oct 20 to 26)

#	Commit	Date (Toronto)	Message
1	c0d1436	2026-10-26 17:23	Add Maven backend skeleton, .env.example and secrets gitignore
2	4e5f716	2026-10-26 17:37	Add AppConfig loader with .env parsing, env override and tests
3	952152a	2026-10-26 21:36	Add LLMClient interface, GeminiClient adapter, MockLLMClient and tests
4	8105e6f	2026-10-26 21:38	Add JobApplication pipeline with State pattern and transition tests
5	eb4749b	2026-10-26 21:44	Add CLI with Command pattern: registry, help and ask commands, with tests

Window 2 (Oct 27 to Nov 2)

#	Commit	Date (Toronto)	Message
1	89da763	2026-10-29 14:28	Add agent core: Tool interface, ToolRegistry, Planner and AgentController with tests
2	83c5e56	2026-10-29 14:30	Add skill extraction with source verification and extract_skills agent tool
3	8b3d343	2026-10-29 14:31	Add job matching with Strategy pattern, JobMatcher and match_job agent tool
4	0087064	2026-10-29 14:33	Add alerts with Observer pattern, scheduler vs user alert sources, and CLI alerts command
5	951d78e	2026-10-29 14:39	Wire agent into CLI: agent command, extract_skills and match_job tools, alert observer
6	2bbafcc	2026-10-29 14:58	Add application tracking service and CLI track command using State pattern
7	bd97f2a	2026-11-02 19:08	Add retry with backoff and readable API errors to GeminiClient, with tests

Window 3 (Nov 3 to 9), code commits

#	Commit	Date (Toronto)	Message
1	724ffd7	2026-11-03 20:46	Add Javalin HTTP server skeleton and shared AppContext
2	a81f85e	2026-11-03 20:50	Add REST endpoints for applications, alerts, matching and agent
3	bbf6454	2026-11-03 21:05	Add skill extraction and deadline endpoints to REST API with tests
4	e3346b1	2026-11-03 21:14	Add tailored document generation with factual source check
5	96d4fec	2026-11-04 15:53	Add mock interview service, REST endpoints, and tests
6	797af78	2026-11-05 00:55	Add Next.js GUI skeleton: deadlines, alerts, skill extraction
7	da3a2cb	2026-11-05 02:32	Add tailored documents page with factual-check warning
8	a6edbda	2026-11-05 14:15	Retry skill extraction once when the model returns no JSON array, with tests
9	05b9b1d	2026-11-05 14:34	Add skills match, tailored documents and mock interview pages; fix Tailwind PostCSS setup
10	bd36e30	2026-11-05 22:55	Add F09 analytics: MetricsService, DashboardMetrics, /api/analytics and CLI stats command, with tests
11	6093bd5	2026-11-05 22:56	Add pipeline GUI page with application tracking and analytics dashboard
12	cfe178f	2026-11-06 12:59	Add F08 bullet optimizer and F07 prep sheet with Factory Method prompt factories, API, CLI and agent tools
13	c4fe438	2026-11-06 12:59	Add bullet optimizer and prep sheet GUI pages
14	6358598	2026-11-07 00:07	Add F10 application package exporter: ExportService, export CLI command and /api/export, with tests
15	0215b83	2026-11-07 16:05	Add export package button to the tailor page
16	ef0edcb	2026-11-08 03:38	Add interview CLI command for F12 and career agent GUI page for F11
17	4037191	2026-11-08 15:12	Add F01 resume PDF import and F02 job posting ingestion with API, CLI, GUI and nav links

Counts were taken from the repository history with `git log` (one command per window, for example `git log --since="2026-11-03 00:00" --until="2026-11-10 00:00" --reverse`), and the source files changed by every commit were listed to decide which commits count.

Checks performed

- **Automated tests:** 230 JUnit tests (`mvn test`). Services run against `MockLLMClient`; the REST layer is tested with Javalin test tools; the CLI commands, the pipeline states, the alert observer, the

matching strategies, the prompt factories and the `GeminiClient` retry (with a scripted `Transport`) have their own tests.

- **Manual runs:** each feature was run in the GUI and in the CLI against the real Gemini API and recorded in the demo videos (Section 7).
- **Secrets:** `.env` is git-ignored; `.env.example` holds placeholders only; no tracked file contains a key-shaped string; a search of the full git history for a Gemini key prefix found nothing.
- **Commit windows:** verified from `git log` as described above.

Important defects found and corrected

Defect	Cause	Fix	Commit
HTTP 429 and 503 from Gemini during live runs.	Rate limits and overload on the first models tried.	Retry with backoff, readable errors, switch to gemini-3.5-flash-lite.	bd97f2a
Skill extraction sometimes failed.	The model returned text without a JSON array.	One retry in `SkillExtractor`, with tests.	a6edbda
GUI loaded without styling.	Tailwind PostCSS setup was missing.	Added the PostCSS configuration.	05b9b1d
AI could claim skills that are not in the resume.	LLM output is not checked by default.	`SourceVerifier` and factual checks (N1, N2).	83c5e56 e3346b1
F09/F10 scripts failed on the first attempt.	The scripts edited files incorrectly.	Reset with git, applied a corrected F10 script.	6358598

Remaining limitations

See the limitations list in Section 1. Full deterministic testing and the KUMA agent-validation work belong to Stage 3.

7. Demonstration index

Each feature has a video showing both the GUI and the CLI, with the timestamp where each part starts. The videos are in `stage2/videos/` of the public repository and open without an access request. F04 and F10 share one video. N1 and N2 appear inside the videos for the features they protect; N3 and N4 are infrastructure used by every demo.

Feature	Video	GUI starts	CLI starts
F01 Resume Import & Parsing	F01-resume-import.mp4	0:00	0:25
F02 Job Posting Ingestion	F02-job-ingestion.mp4	0:00	0:37
F03 Skill Gap & Match Analysis	F03-skill-match.mp4	0:00	0:42
F04 Tailored Cover Letter	F04-F10-cover-letter-and-export.mp4	0:00	0:58
F05 Pipeline State Management	F05-pipeline.mp4	0:00	0:53
F06 Deadline Tracking & Alerts	F06-deadlines-alerts.mp4	0:00	0:36
F07 Interview Prep Sheet	F07-prep-sheet.mp4	0:00	0:27
F08 Bullet Point Optimizer	F08-bullet-optimizer.mp4	0:00	0:30
F09 Analytics Dashboard	F09-analytics.mp4	0:00	0:30
F10 Application Package Exporter	F04-F10-cover-letter-and-export.mp4	0:00	0:58
F11 Career Agent Assistant	F11-career-agent.mp4	0:00	0:24
F12 Mock Interview Practice	F12-mock-interview.mp4	0:00	1:11